

05 - Grafos

Diplomado Data Engineer
Universidad de Santiago de Chile



UNIVERSIDAD
DE SANTIAGO
DE CHILE

Juan Pablo Castillo

Grafos

- Los grafos es la estructura de datos más flexible que estudiaremos.
- Listas: único sucesor y predecesor
- Árboles: pueden tener más de un sucesor, pero un único predecesor
- Grafos: veremos que permiten tener varios sucesores como también varios predecesores
- Los grafos son muy útiles ya que nos permiten modelar muchos problemas de la vida real.

Grafos

Aplicaciones

- Modelar conectividad en redes de computadores y de comunicación.
- Representar mapas como un conjunto de localidades con distancias entre ellas.
- Modelar capacidades de flujos en redes de transporte.
- Encontrar un camino desde una condición inicial hasta una condición final (muy usado en la inteligencia artificial).
- Representar redes sociales.
- Representación de la web.
- Etc.

Grafos

Aplicación: Redes Sociales



Grafos

Terminología

Un grafo $G = (V, E)$ consiste en:

- Un conjunto de vértices $V = \{v_0, v_1, \dots, v_{n-1}\}$

- Un conjunto de arcos (o aristas)

$E = \{(v_i, v_j) \mid \text{existe conexión entre los vértices } v_i \text{ y } v_j \text{ del grafo}\}$

- Cada arco en E es una conexión entre pares de vértices de V .

- Al dibujar un grafo:

- Vértices son puntos en el plano
- Arcos son líneas entre pares de vértices

Grafos

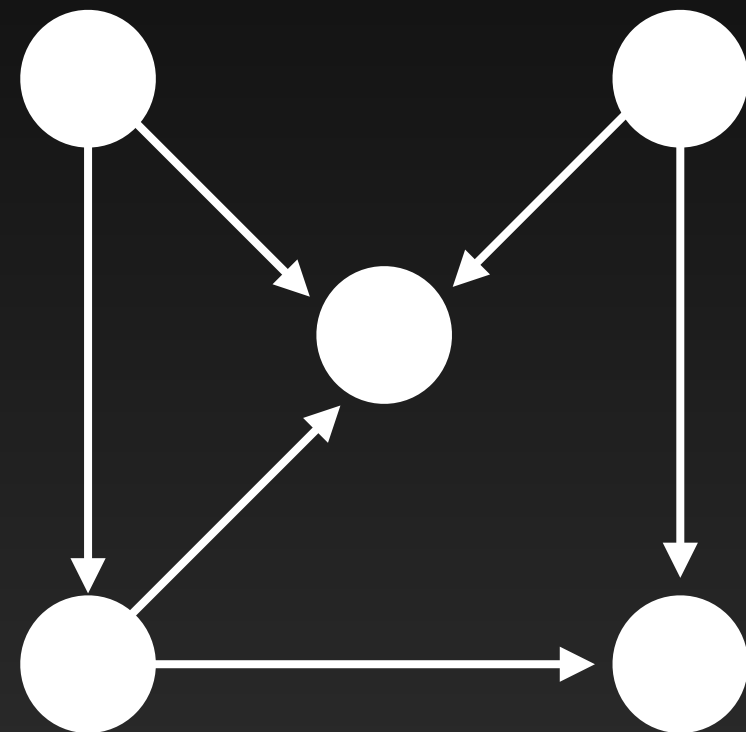
Terminología

- Un grafo con (relativamente) **pocos arcos** se conoce como **ralo** o **disperso**.
- Un grafo con **muchos arcos** como **denso**.
- Un grafo **completo** tiene todos los posibles arcos.
- Generalmente:
 - **Los vértices se usan para representar los datos**
 - **Los arcos se usan representar relaciones entre los datos**
- Ejemplos:
 - Los vértices representar las personas registradas en una red social.
 - Los arcos representan una relación de amistad entre personas.

Grafos: Grafos Dirigidos

Terminología

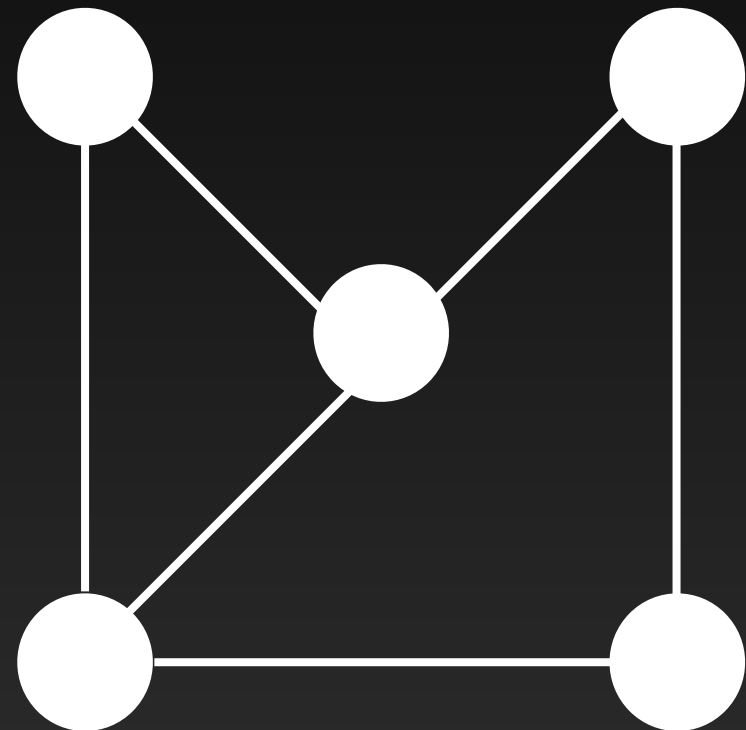
- Cuando la dirección de un arco es relevante, hablamos de **grafos dirigidos** (o simplemente dígrafos).
- Los arcos se dibujan con flechas (direcciones)
- Ejemplo:
 - Seguidores en *instagram* o *X*



Grafos: Grafos No Dirigidos

Terminología

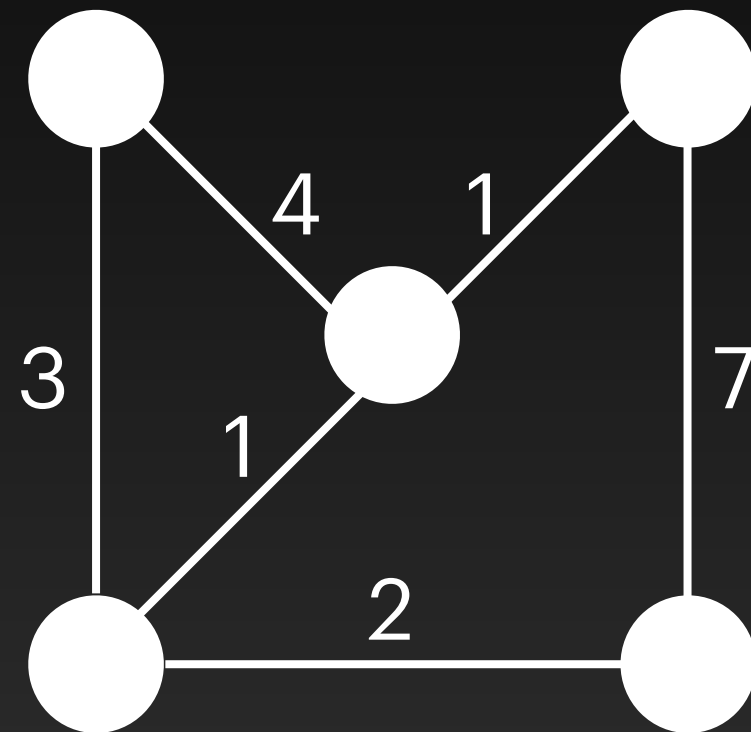
- Cuando la relación entre vértices es en ambos sentidos, hablamos de grafos no dirigidos (o simplemente grafos).
- Los arcos se dibujan como líneas sin dirección.
- Ejemplo:
 - El concepto de amistad en Facebook, se da en ambas direcciones.



Grafos: Grafos con Peso

Terminología

- Cuando a cada arco se le asocia un peso (generalmente un valor en los reales), el grafo se llama pesado.
- Los arcos se dibujan con el peso asociado sobre ellos.
- Ejemplo:
 - Costos de comunicación en redes.
 - Distancias o tiempos de desplazamiento en mapas.



Grafos

Terminología

- Si (v_i, v_j) es un arco de E entonces v_i es **adyacente** a v_j .
- También se les conoce como **vértices vecinos**.
- El arco (v_i, v_j) es **incidente** en los vértices v_i y v_j .
- El **grado** o **valencia** de un vértice es la cantidad de arcos que inciden en él.
 - En dígrafos se distingue entre **grado de entrada** y **grado de salida**.

Grafos

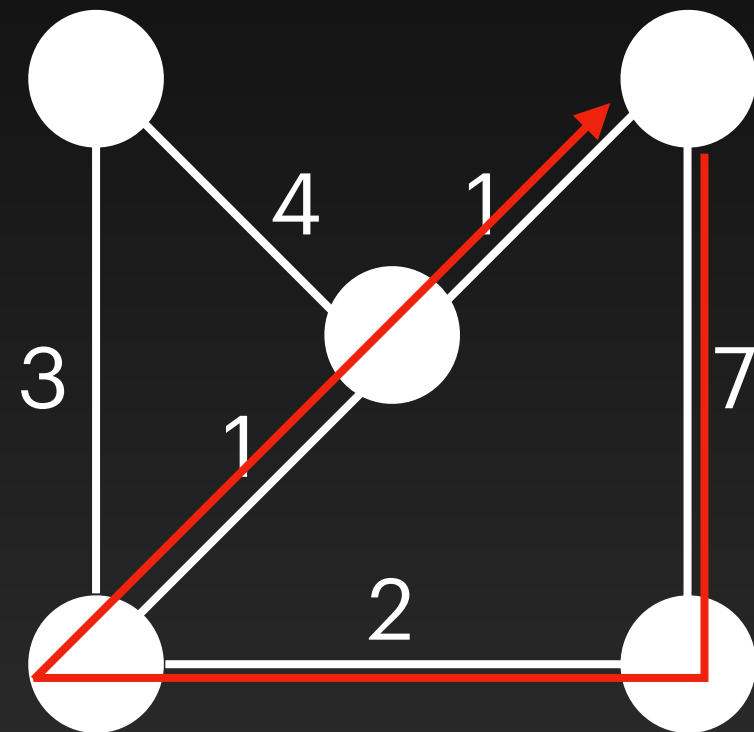
Terminología

- Dada una secuencia de vértices $\langle a_0, a_1, \dots, a_{m-1} \rangle$ diremos que es un camino de largo $m - 1$ si $(a_i, a_{i+1}) \in E$ para todo i en la secuencia.
- **Camino cerrado:** un camino es cerrado si el primer y el último vértice son el mismo.
- **Camino simple:** un camino es simple si todos los vértices son distintos (excepto quizás el primero y el último)
- **Largo del camino:** es la cantidad de arcos que contiene.
- **Peso de un camino:** es la suma de los pesos de los arcos que componen el camino.

Grafos

Terminología

- **Ciclo:** es un camino cerrado simple.
- Un grafo que no contiene ciclos se llama **acíclico**.
- Ejemplo:
 - Ciclo de largo 4 y peso 11



Grafos

Terminología

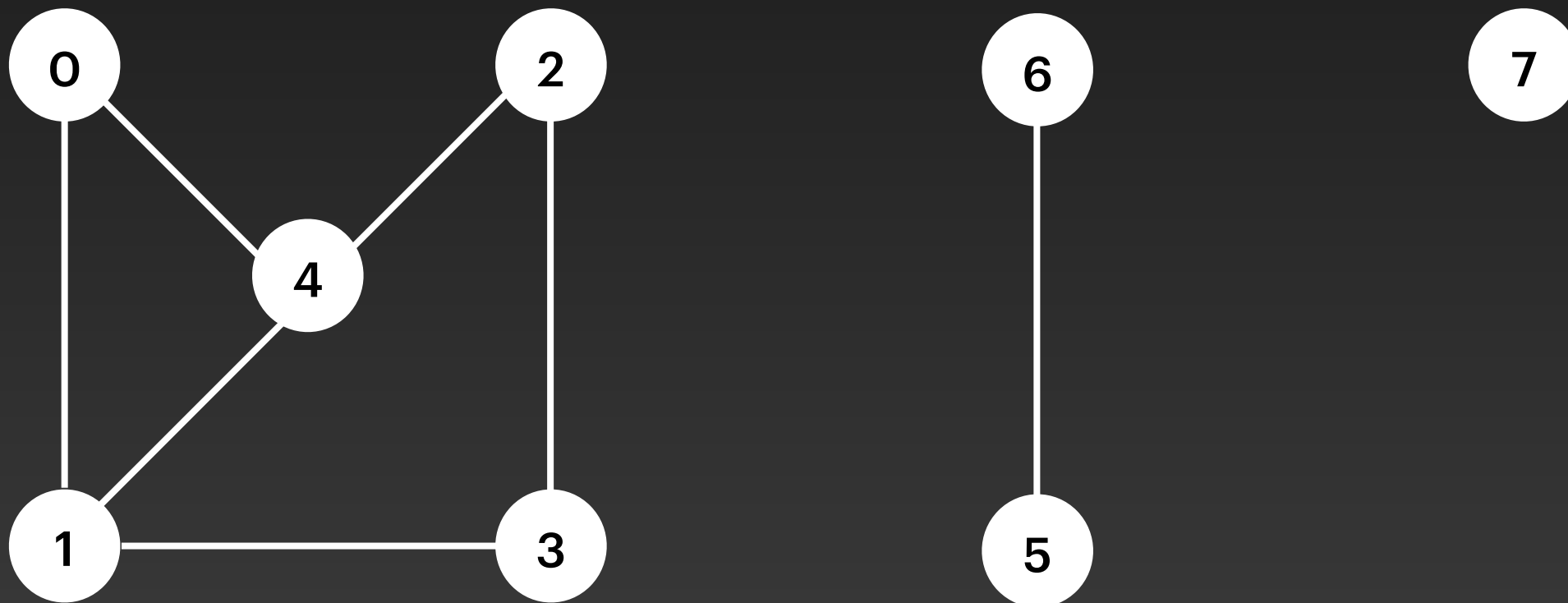
Dado un grafo $G = (V, E)$, definimos un **subgrafo** $S = (V', E')$ tal que:

- V' es subconjunto de V , y
- E' es subconjunto de E sobre los vértices en V'
- Es decir, ese subconjunto E' debe tener solo vértices de V' .

Grafos

Terminología

- Un grafo (no dirigido) es **conexo** si hay al menos un camino entre cualquier par de vértices del mismo.
- Si un grafo no es conexo, lo forman sus componentes conexas, que son los subgrafos conexos máximos.



Búsquedas y Recorridos en Grafos

Grafos

En muchas aplicaciones es útil visitar los vértices de un grafo en algún orden en específico, basado en la **topología del grafo**.

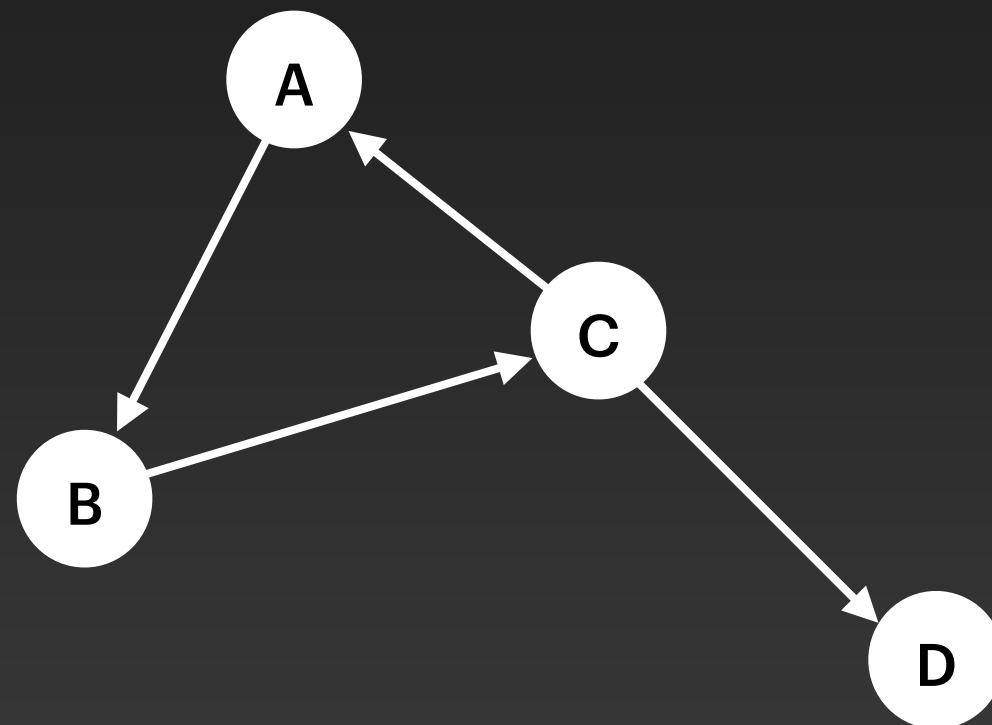
- Esto es similar a recorrer los nodos de un árbol en algún orden dado.
- Este concepto se conoce como **recorrido de grafos**.
- Al igual que en los árboles, hay diferentes maneras de recorrer grafos
 - Por ejemplo en inteligencia artificial, uno quiere moverse desde un estado inicial a un estado final, siguiendo las conexiones.
 - Hay que recorrer los vértices y arcos en algún orden para lograrlo.

Grafos

- Típicamente, un recorrido comienza en un vértice y trata de alcanzar los demás vértices del grafo.
- Sin embargo, pueden presentarse inconvenientes:
 - Algunos vértices pueden ser inalcanzables desde algunos otros vértices
 - El grafo no es conexo
 - Algunos vértices pueden ser visitados múltiples veces siguiendo distintos caminos.

Grafos

- Por ejemplo, si en el siguiente dígrafo un quiere ir desde el vértice A al vértice D , se corre el riesgo de caer infinitamente en el ciclo $A \rightarrow B \rightarrow C$, si los vértices no se marcan adecuadamente como “ya visitados”.



**Al marcar los
vértices visitados,
se evitan ciclos
infinitos**

Grafos: Recorridos

Recorrido en Profundidad (Depth First Search - DFS)

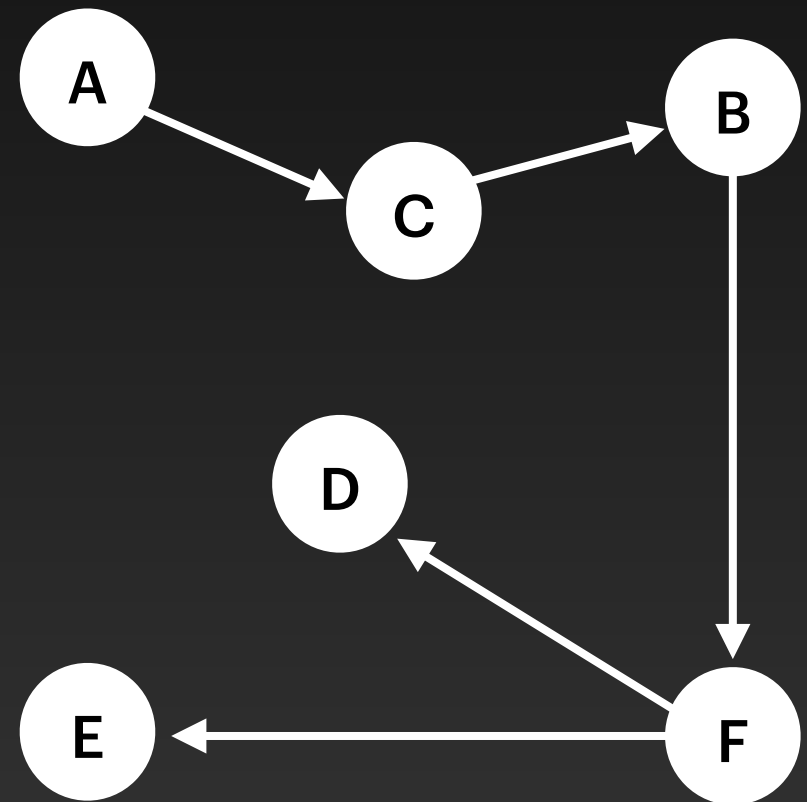
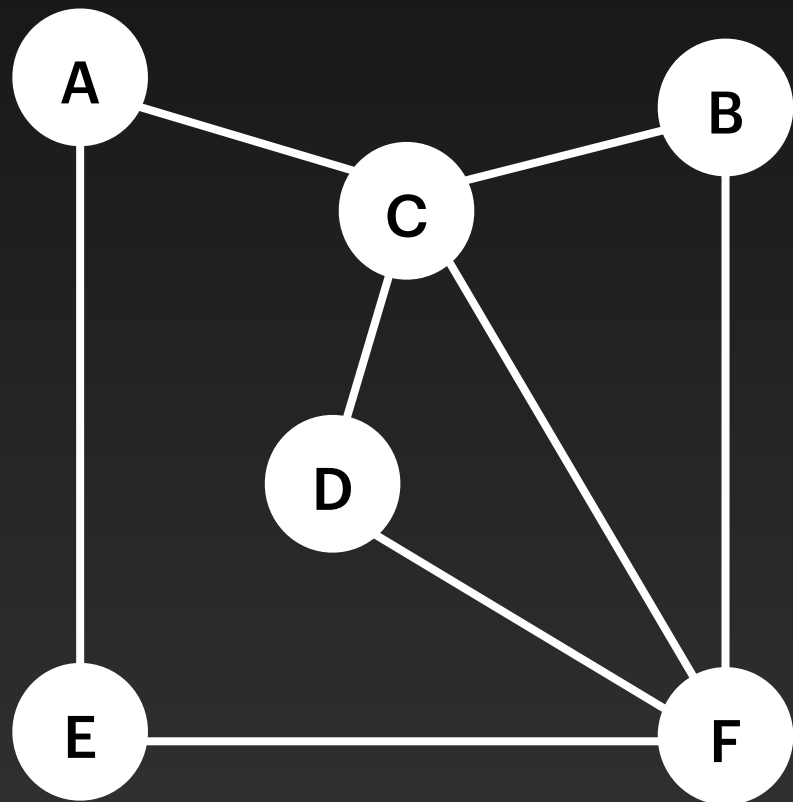
1. Se procesa un nodo inicial v arbitrario
 - Puede ser cualquiera del grafo.
2. Luego se procesa cada uno de los nodos vecinos a v *que aún no son visitados*, iniciando un DFS desde cada uno de ellos.
3. Cuando todos los vecinos de un vértice hayan sido visitados, se retrocede al último vértice visitado que le queden vecinos sin procesar.

Finalmente todos los vértices alcanzables desde el vértice inicial v serán visitados, con lo que el algoritmo finaliza.

Grafos: Recorridos

Recorrido en Profundidad (Depth First Search - DFS)

El siguiente ejemplo muestra un grafo y el correspondiente recorrido DFS a partir del vértice A.



Importante: el resultado de un recorrido DFS en un grafo es un árbol

Grafos:Recorridos

Recorrido en Profundidad (Depth First Search - DFS)

- El tiempo total del algoritmo es $O(|V| + |E|)$
- Todos los arcos y vértices son visitados una única vez

Grafos: Recorridos

Recorrido en Amplitud (Breadth-First Search - BFS)

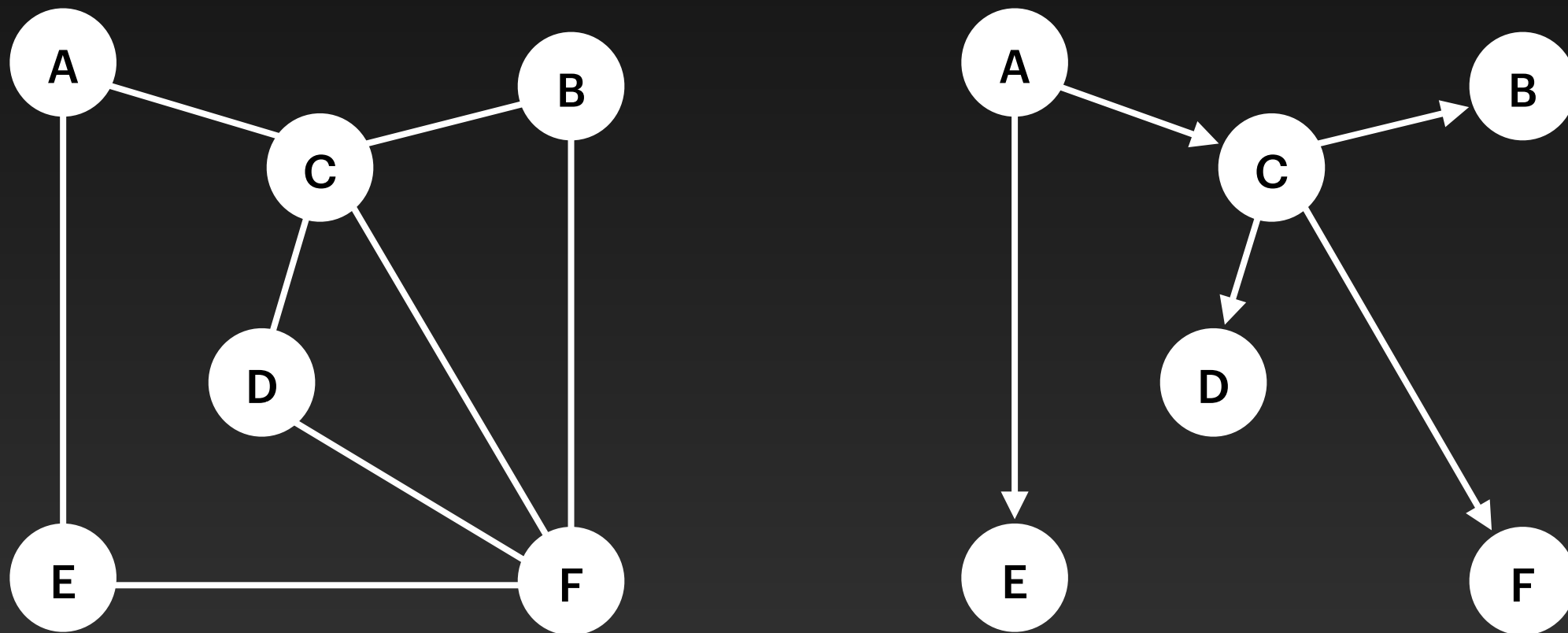
1. Se procesa un nodo inicial v arbitrario
2. Luego se procesa cada uno de los nodos vecinos de v que aún no son visitados
3. Una vez procesados todos los vecinos, se repite el proceso con los nodos adyacentes del primer vecino, y así siguiendo hasta visitar todos los nodos

Se usa una cola como estructura de datos auxiliar para realizar este recorrido (note la similitud con los árboles)

Grafos: Recorridos

Recorrido en Amplitud (Breadth-First Search - BFS)

El siguiente ejemplo muestra un grafo y el correspondiente recorrido BFS a partir del vértice A.



Importante: el resultado de un recorrido BFS en un grafo es un árbol

Grafos: Recorridos

Recorrido en Amplitud (Breadth-First Search - BFS)

- El tiempo total del algoritmo es $O(|V| + |E|)$
- Todos los arcos y vértices son visitados una única vez

Problema del Camino más Corto

Grafos: Camino más Corto

- En un mapa de rutas entre ciudades, por ejemplo, a la ruta que une dos ciudades generalmente se le asocia un peso que indica la distancia (o costo) de la ruta.
- Dadas dos ciudades del mapa, ¿Cuál es la ruta de menor costo entre esas dos ciudades?

Definición formal:

- Dado un grafo con pesos, se quiere conocer la longitud del camino de menor peso entre un vértice de inicio y resto de vértices.
- Notar que “más corto” aquí significa de “menor peso”
- Dado que hay múltiples caminos entre vértices, el problema es no trivial

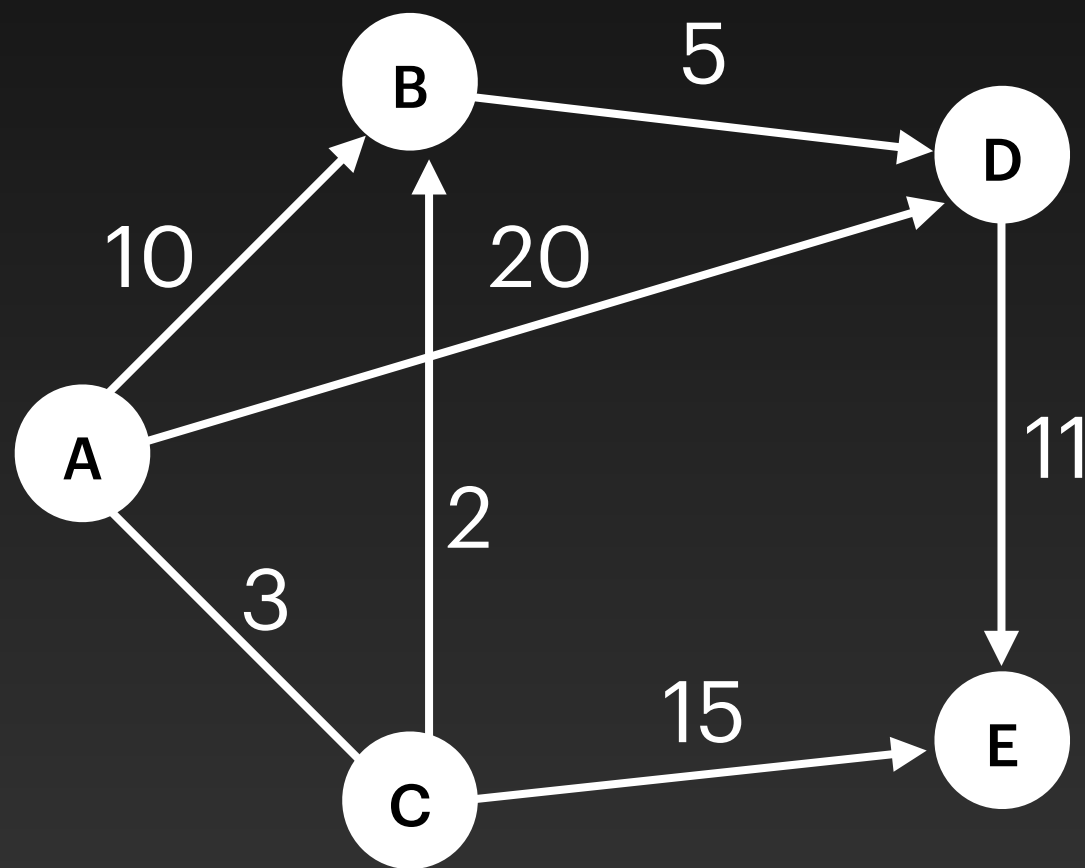
Grafos: Camino más Corto

Aplicaciones

- Ruteo en redes
- Redes de transporte
- Redes sociales
- Sistemas de navegación
- Control de robots

Grafos: Camino más Corto

Por ejemplo, consideremos el siguiente grafo con pesos:



Camino más corto:

$A \rightarrow B$: costo 5

$A \rightarrow C$: costo 3

$A \rightarrow D$: costo 10

$A \rightarrow E$: costo 18

Grafos: Camino más Corto

Algoritmo de Dijkstra

- E. Dijkstra descubrió el un algoritmo que resuelve este problema en 1956 pero no fue hasta 1959 cuando lo publicó.
- El algoritmo encuentra el camino mínimo entre un vértice s al resto de los vértice del grafo.
- Si bien podríamos estar interesados en un par de vértices, Dijkstra demostró que no hay mejor forma de hacerlo.
- El tiempo del algoritmo es $O((|V| + |E|) \cdot \lg |V|)$

Problema del Árbol Recubridor Mínimo

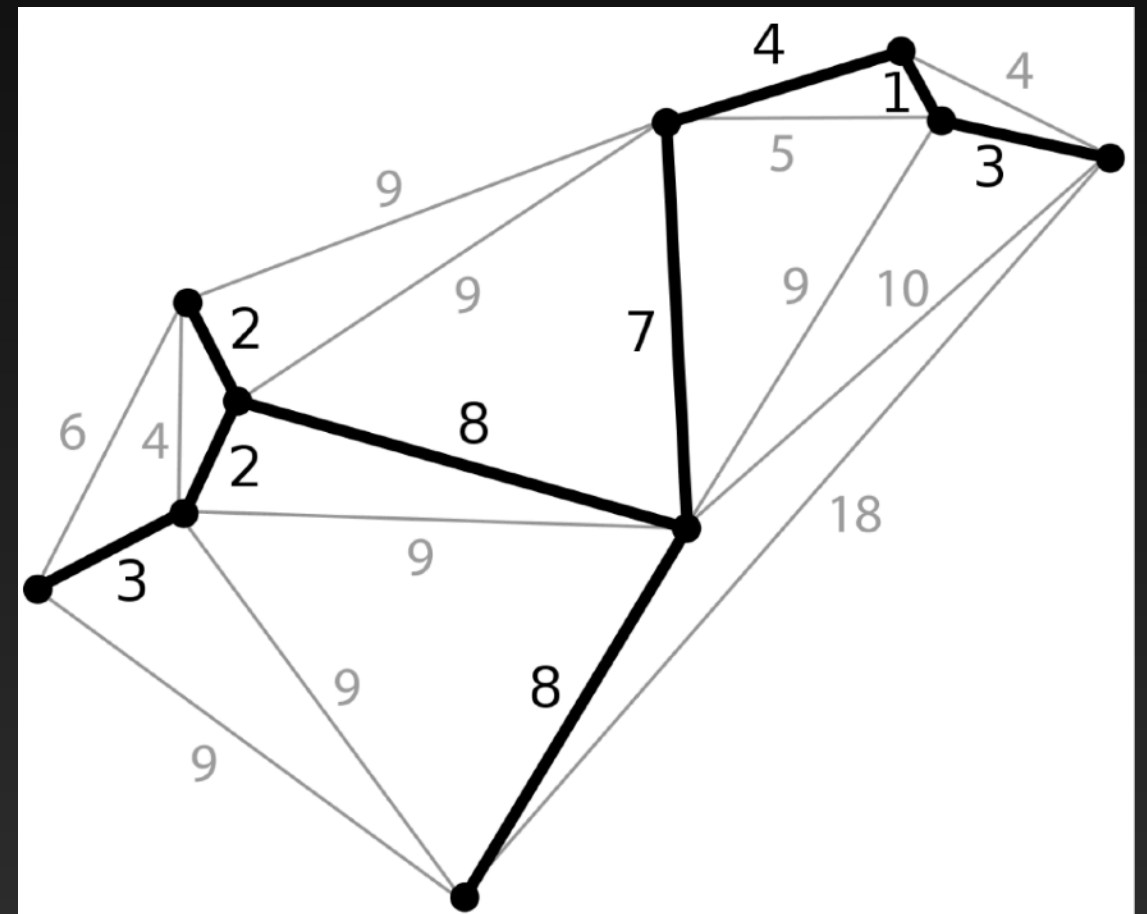
Grafos: Árbol Recubridor Mínimo

- En un grafo, es común encontrar caminos redundantes
 - i.e., *es posible llegar desde un nodo a otro mediante diferentes caminos.*
- Si en un grafo conexo se eliminan caminos redundantes, el grafo sigue siendo conexo.
- **Árbol:** es un grafo conexo sin ciclos
- **Árbol Recubridor:** Dado un grafo conexo $G = (V, E)$, un árbol recubridor es un subgrafo de G con el mismo conjunto de vértices V tal que es un árbol.

Grafos: Árbol Recubridor Mínimo

Arbol Recubridor Mínimo:

- Dado un grafo conexo con pesos $G = (V, E)$, su árbol recubridor mínimo es el árbol recubridor de G cuya suma de pesos asociados a los arcos es mínimo.
- ***Para un grafo en particular, pueden existir varios árboles recubridores de costo mínimo diferentes***



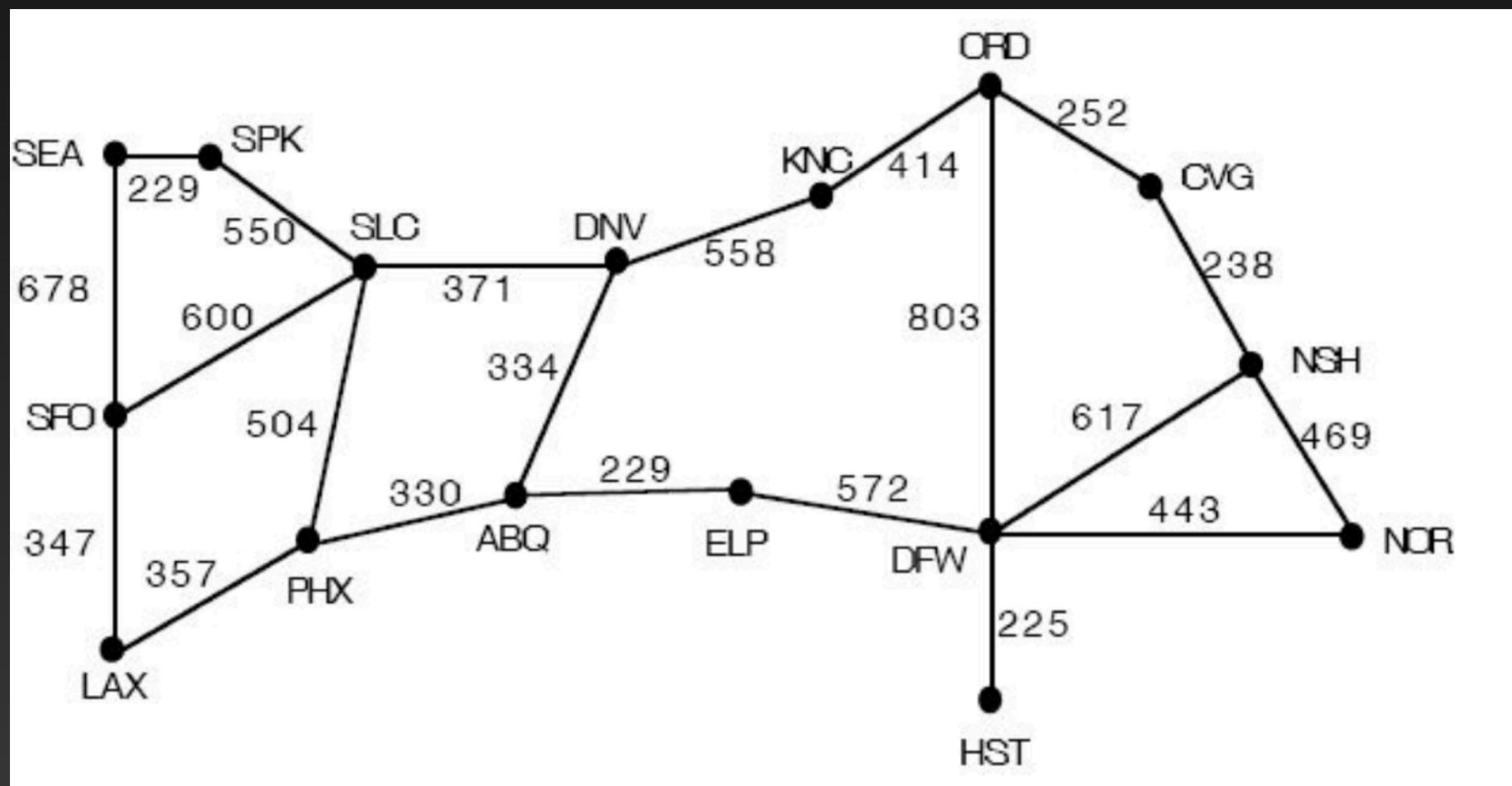
Grafos: Árbol Recubridor Mínimo

Aplicaciones

- Diseño de redes
- Tratamiento de imágenes
- Reconocimiento facial
- Agrupamiento de vértices en grafos
- Etc

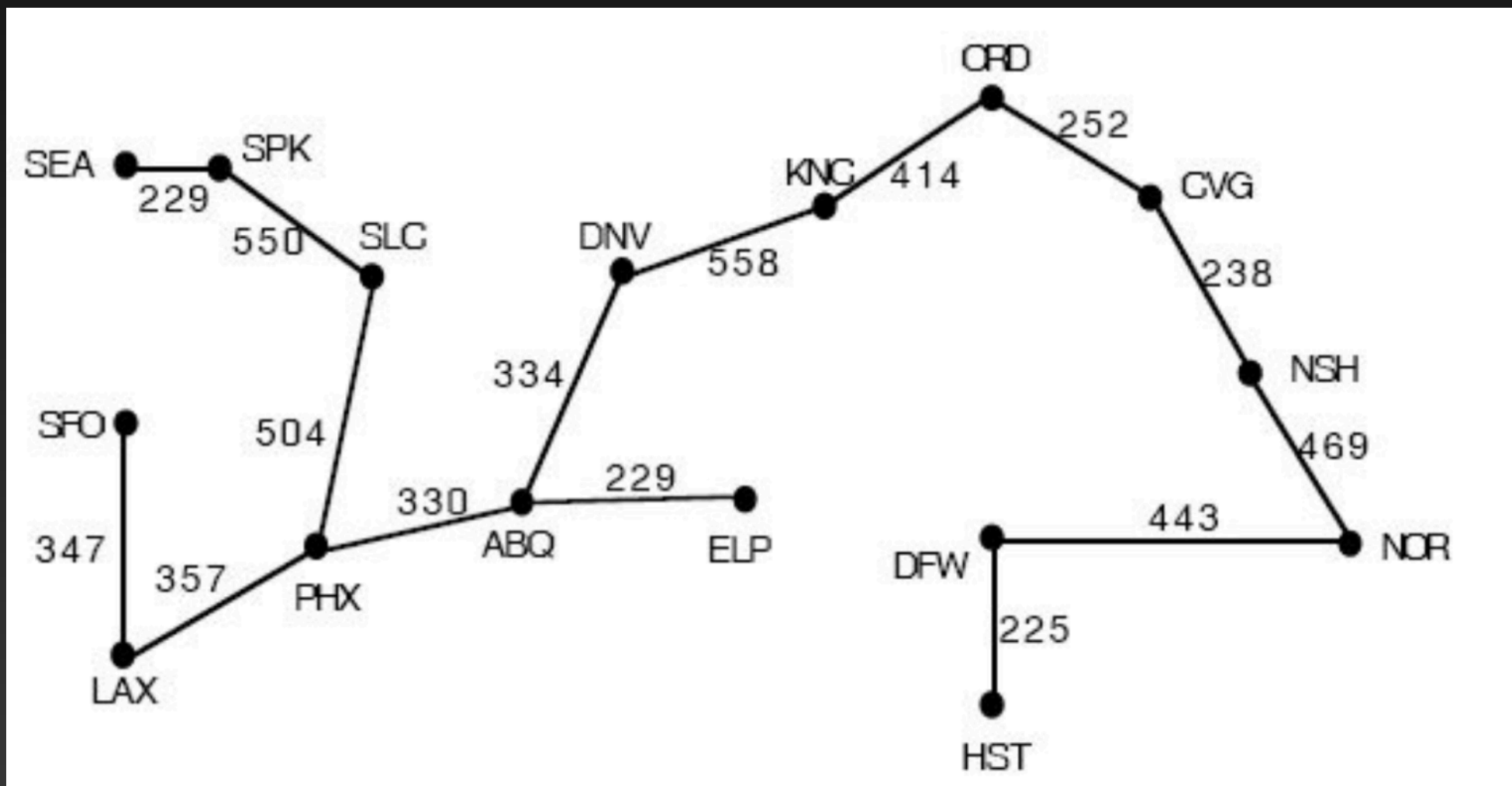
Grafos: Árbol Recubridor Mínimo

Supongamos el siguiente grafo que modela una red con algunos aeropuertos en Estados Unidos, y el costo de comunicación entre ellos (ya sea en cuanto tendido de cables, mantenimiento, etc).



Grafos: Árbol Recubridor Mínimo

Este sería el árbol recubridor mínimo, con costo total de 5479



Grafos: Árbol Recubridor Mínimo

- Hay dos algoritmos que resuelven este problema:
 - **Algoritmo de Prim**
 - También conocido como Algoritmo de DJP (por quienes descubrieron y redescubrieron el algoritmo)
 - Dijkstra, Jarnik, y Prim.
 - Tiempo de ejecución: $O((|V| + |E|) \cdot \lg |V|)$
 - **Algoritmo de Kruskal**
 - Tiempo de ejecución: $O(|E| \cdot \lg |E|)$